Documento de apoyo (Sesión 8, Unidad 2)

Orquestación de Pipelines con Apache Airflow

DAGs, automatización, dbt y operación de procesos de datos

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Septiembre de 2026

Resumen

Un pipeline de datos confiable necesita coordinar procesos, respetar dependencias y responder de forma controlada ante errores. Esta sesión estudia Apache Airflow como plataforma de orquestación de flujos de trabajo: DAGs, tareas, operadores, programación y seguimiento de ejecuciones. Se relaciona la automatización con el modelado analítico en dbt y las cargas hacia un warehouse. El documento combina fundamentos, ejemplos de Python y SQL, una guía de despliegue local con Docker aplicada al caso SGFood. La planificación, la responsabilidad operacional, la disciplina técnica y el pensamiento crítico orientan las decisiones de diseño y recuperación.

Palabras clave: Apache Airflow, orquestación, pipeline, DAG, task, operator, scheduler, executor, dbt, SQL, Docker, monitoreo, idempotencia.

Índice

1. Introducción **4**
1.1. Alcance y versión de los ejemplos 4

2. Marco formativo de la sesión **5**
2.1. Área actitudinal: saber ser 5
2.2. Área de conocimiento: saber 5
2.3. Área de habilidades: saber hacer 6

3. Fundamentos de orquestación de pipelines **7**
3.1. Pipeline, automatización y orquestación 7
3.2. ETL y ELT bajo un mismo coordinador 7
3.3. Decidir qué automatizar 7

4. Conceptos y arquitectura de Apache Airflow **8**
4.1. DAG, tarea y operador 8
4.2. Componentes y responsabilidades 8
4.3. Metadatos y datos de negocio 8

5. Creación de DAGs y definición de dependencias **9**
5.1. Estructura de un archivo 9
5.2. Orden explícito y paralelismo 10
5.3. Separar construcción y ejecución 10

6. Programación, intervalos y reintentos **11**
6.1. Calendario y zona horaria 11
6.2. Intervalo de datos y fecha de ejecución 11
6.3. Reintentos y límites 11

7. Ejemplo guiado: pipeline ETL de ventas **13**
7.1. Objetivo y contrato de datos 13
7.2. Archivo completo: s8_ventas.py 13
7.3. Lectura del comportamiento 15

8. Integración con dbt, SQL y el warehouse **16**
8.1. Responsabilidades y orden de ejecución 16
8.2. Ejecutar un proyecto dbt preparado 16

8.3. Consultas y validación SQL 17
8.4. Carga idempotente al warehouse 17

9. Variables, conexiones y comunicación entre tareas 19
9.1. Configuración con Variables 19
9.2. Conexiones y credenciales 19
9.3. XCom y almacenamiento compartido 19

10. Monitoreo, alertas y manejo de fallos 20
10.1. Interpretar los estados 20
10.2. Logs y evidencia 20
10.3. Alertas y callbacks 20
10.4. Procedimiento de recuperación 20

11. Despliegue local con Docker 22
11.1. Preparación del entorno 22
11.2. Comprobación y primera ejecución 22
11.3. Prueba de fallo y recuperación 23
11.4. Diagnóstico y cierre 23

12. Buenas prácticas operacionales 24
12.1. Diseñar para repetición y trazabilidad 24
12.2. Control de recursos y publicación 24
12.3. Lista de revisión del pipeline 24

13. Conclusiones 25

1. Introducción

En la semana anterior se abordó la construcción de modelos analíticos con dbt. Sin embargo, ejecutar correctamente un modelo una vez no resuelve su operación diaria. En SGFood, por ejemplo, el reporte de ventas necesita recibir archivos de las sucursales, revisar su contenido, cargar datos y actualizar las tablas analíticas antes de que el equipo consulte los resultados. Si una sucursal entrega información incompleta, el sistema debe detectar esa condición y evitar publicar métricas engañosas.

La orquestación hace explícitos el orden, la frecuencia y las condiciones de ejecución. Un script puede transformar un archivo; el flujo que lo contiene debe establecer cuándo está disponible ese archivo, qué tarea lo consume y qué sucede si la transformación falla. Separar estas responsabilidades facilita explicar y operar el proceso.

Apache Airflow representa flujos como grafos dirigidos acíclicos y mantiene información sobre sus ejecuciones. Permite combinar tareas de Python, comandos y herramientas externas mediante dependencias declaradas en código [1]. En este documento se estudia su aplicación a procesos de datos por lotes.

1.1. Alcance y versión de los ejemplos

Los ejemplos utilizan la API pública de **Airflow 3** y toman **3.1.0** como referencia reproducible para el laboratorio. Esta selección identifica una versión concreta; no afirma que sea la más reciente. Se emplean importaciones desde `airflow.sdk` y operadores de paquetes *provider*. Los materiales escritos para Airflow 2 pueden usar rutas de importación y servicios diferentes.

El primer DAG es una demostración pequeña y autocontenida de extracción, transformación, validación y carga de un archivo. La integración con dbt y SQL se presenta después como una ampliación que requiere preparar el proyecto, sus dependencias y la conexión al warehouse. No se presupone que esas configuraciones existan automáticamente.

2. Marco formativo de la sesión

2.1. Área actitudinal: saber ser

Valor	Aplicación en la sesión
Planificación	Se fortalece al diseñar flujos de trabajo ordenados, programados y dependientes entre sí.
Responsabilidad operacional	Se promueve al monitorear ejecuciones, registrar errores y definir acciones ante fallas de pipelines.
Disciplina técnica	Se desarrolla al estructurar DAGs, tareas y dependencias siguiendo buenas prácticas de automatización.
Pensamiento crítico	Se aplica al decidir qué procesos deben automatizarse, con qué frecuencia y bajo qué condiciones.

Estos valores se evidencian mediante decisiones verificables: un diagrama previo expresa planificación; un registro de recuperación demuestra responsabilidad; nombres claros y código versionado reflejan disciplina; y justificar la frecuencia de carga ejercita el pensamiento crítico.

2.2. Área de conocimiento: saber

Competencia: Diseña y automatiza pipelines de datos mediante Apache Airflow, integrando tareas de extracción, transformación, validación y carga para coordinar procesos de datos de forma programada, monitoreable y reproducible.

Tema	Subtema
Orquestación de pipelines	Conceptos de DAG, task, operator, scheduler y executor.
Apache Airflow	Creación de DAGs y definición de dependencias.
Automatización de procesos de datos	Programación, reintentos, monitoreo y manejo de fallos.
Integración con dbt y bases de datos	Ejecución de modelos dbt, consultas SQL y cargas hacia el warehouse.
Buenas prácticas operacionales	Variables, conexiones, logs, alertas y despliegue local con Docker.

2.3. Área de habilidades: saber hacer

Se aplica la misma competencia mediante el diseño, ejecución y recuperación de un pipeline. Al finalizar, el estudiante debe poder explicar sus dependencias, interpretar el estado de una tarea, comprobar los datos producidos y repetir una carga sin generar duplicados.

3. Fundamentos de orquestación de pipelines

3.1. Pipeline, automatización y orquestación

Un pipeline es una secuencia organizada de etapas que produce un resultado de datos. La automatización permite ejecutar una operación sin repetir manualmente sus instrucciones. La orquestación coordina varias operaciones considerando dependencias, condiciones y recursos.

Considérese este caso de diseño: SGFood recibe ventas por sucursal. Un comando que importa un CSV automatiza la carga. Para disponer de un reporte diario también se debe comprobar la llegada del archivo, validar las ventas, actualizar modelos y registrar la ejecución. El diseño del pipeline debe responder qué operaciones son obligatorias y qué salida se considera publicable.

3.2. ETL y ELT bajo un mismo coordinador

En ETL, la transformación precede a la carga al destino analítico. En ELT, los datos se cargan primero a una zona inicial del warehouse y se transforman dentro de él. Airflow puede coordinar ambos enfoques; el trabajo concreto lo realiza el código o sistema invocado por cada tarea.

ETL: Extraer → Transformar → Validar → Cargar

ELT: Extraer → Cargar raw → dbt → Verificar

3.3. Decidir qué automatizar

La frecuencia debe responder a la disponibilidad del origen y a la necesidad de negocio. Para un reporte utilizado cada mañana, una actualización diaria puede ser suficiente. Ejecutar el mismo proceso cada minuto, cuando el archivo cambia una vez al día, consume recursos sin mejorar el resultado.

Antes de definir un flujo, se propone responder: ¿qué evento habilita el trabajo?, ¿qué datos identifica cada ejecución?, ¿cuál es el plazo útil del resultado?, ¿qué comprobación impide publicar datos incorrectos? y ¿quién atiende los errores? Estas preguntas convierten una secuencia de comandos en un proceso operable.

Airflow se orienta a workflows finitos por lotes. Si el problema requiere procesar continuamente cada evento, conviene separar el motor de streaming de los trabajos de preparación, verificación o consolidación que lo rodean.

4. Conceptos y arquitectura de Apache Airflow

4.1. DAG, tarea y operador

Un **DAG** es un grafo dirigido sin ciclos: los nodos representan tareas y las aristas, dependencias. Una **task** es una unidad de trabajo del flujo; un **operator** aporta una implementación reutilizable para un tipo de operación. Una función decorada con `@task` permite expresar trabajo Python con TaskFlow [1], [5].

En el caso propuesto, `extraer` puede preceder a `transformar`, y `validar` debe preceder a `cargar`. Agregar una dependencia desde cargar hacia extraer cerraría un ciclo. La repetición diaria se resuelve creando nuevas ejecuciones del DAG, no dibujando una flecha de regreso.

4.2. Componentes y responsabilidades

Componente	Responsabilidad
DAG processor	Procesa los archivos que definen los flujos.
Scheduler	Decide cuándo corresponde programar tareas según el calendario y sus dependencias.
Executor	Determina el mecanismo con el que se ejecuta el trabajo programado.
Workers	Ejecutan las tareas; su distribución depende del executor.
API server e interfaz	Permiten consultar y administrar ejecuciones en el despliegue de Airflow 3.
Base de metadatos	Conserva información operacional de los flujos y sus estados.
Triggerer	Atiende esperas de tareas diferidas cuando se utilizan operadores que las soportan.

La separación entre programación y ejecución permite distribuir trabajo. El entorno Docker oficial de referencia utiliza CeleryExecutor, workers y Redis como intermediario de mensajes [1], [6].

4.3. Metadatos y datos de negocio

La base interna de Airflow mantiene información de operación. El warehouse almacena las ventas, dimensiones y modelos utilizados para análisis. Aunque ambos puedan usar PostgreSQL, cumplen funciones distintas. En la práctica del curso se deben identificar con nombres y conexiones separados para evitar cargar ventas en la base de metadatos.

5. Creación de DAGs y definición de dependencias

5.1. Estructura de un archivo

Un archivo de DAG contiene importaciones, configuración, declaraciones de tareas y relaciones. Con TaskFlow, llamar una función decorada mientras se construye el DAG crea una tarea y una referencia a su resultado; su cuerpo se ejecuta posteriormente al correr esa tarea [5].

Listing 1: DAG mínimo para Airflow 3

```python
import pendulum
from airflow.sdk import dag, task

@dag(
    dag_id="s8_saludo",
    schedule=None,
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    catchup=False,
    tags=["semana8"],
)
def saludo():
    @task
    def preparar():
        return "SGFood"

    @task
    def mostrar(nombre):
        print(f"Pipeline preparado para {nombre}")

    mostrar(preparar())

saludo()
```

Guardar el archivo en `dags/s8_saludo.py` permite que el procesador lo descubra. Como `schedule=None`, se inicia manualmente. El resultado de preparar alimenta mostrar y establece una dependencia implícita.

5.2. Orden explícito y paralelismo

La expresión `a >> b` exige que b espere a a según su regla de disparo. Con la regla predeterminada, las tareas previas deben terminar correctamente. Para dos extracciones independientes puede utilizarse:

```
[extraer_ventas, extraer_productos] >> integrar
integrar >> validar >> publicar
```

Este fragmento supone objetos de tareas previamente declarados. Las extracciones son elegibles para ejecutarse en paralelo; la concurrencia real también depende de los recursos. Numerar nombres de archivos o escribir una tarea primero en Python no sustituye una dependencia.

5.3. Separar construcción y ejecución

No se deben descargar archivos ni consultar bases al importar el DAG. Ese código puede evaluarse repetidamente durante su procesamiento. Las operaciones pertenecen al cuerpo de tareas; esto evita efectos secundarios y reduce el costo de descubrimiento [2].

6. Programación, intervalos y reintentos

6.1. Calendario y zona horaria

Un calendario expresa cuándo se generan ejecuciones programadas. Es conveniente declarar una zona horaria explícita y una fecha inicial fija. `schedule=None` es útil durante el aprendizaje porque permite controlar cada prueba.

Configuración	Interpretación
`schedule=None`	Sin creación automática de ejecuciones por calendario.
`0 6 * * *`	Calendario cron a las 06:00 cada día, en la zona del DAG.
`0 */4 * * *`	Calendario cron cada cuatro horas, al minuto cero.
`catchup=False`	Evita crear automáticamente todo el historial pendiente.
`max_active_runs=1`	Limita el DAG a una ejecución activa a la vez.

6.2. Intervalo de datos y fecha de ejecución

El tiempo de inicio real y el periodo procesado son conceptos distintos. Para ilustrar intervalos diarios se propone usar explícitamente `CronDataIntervalTimetable`; así no se depende de la configuración que selecciona el comportamiento de una cadena cron [4], [9].

```
from airflow.timetables.interval import CronDataIntervalTimetable

# Argumentos para @dag(...)
schedule= CronDataIntervalTimetable(
    "0 0 * * *", timezone="America/Guatemala"
)
```

El intervalo que comienza el 9 de marzo a las 00:00 y termina el 10 a las 00:00 se programa después de terminar ese periodo. Los filtros deben usar inicio inclusivo y fin exclusivo: fecha $\geq$ inicio y fecha $<$ fin. Si la tarea arranca tarde, continúa procesando el mismo intervalo.

6.3. Reintentos y límites

`retries=2` permite dos reintentos después del intento inicial. `retry_delay` establece la espera base; el retroceso exponencial aumenta esa espera. `execution_timeout` limita la duración de cada intento de una tarea [8].

```
from datetime import timedelta
```

```
2
3 default_args = {
4     "retries": 2,
5     "retry_delay": timedelta(minutes=1),
6     "retry_exponential_backoff": True,
7     "execution_timeout": timedelta(minutes=10),
8 }
```

Una interrupción temporal de red puede desaparecer en el siguiente intento. Un archivo con columnas incorrectas normalmente necesita una corrección. En el ejemplo práctico la validación tendrá cero reintentos: volver a revisar los mismos datos defectuosos no los modifica.

7. Ejemplo guiado: pipeline ETL de ventas

7.1. Objetivo y contrato de datos

El siguiente ejemplo original extrae tres ventas incorporadas en el código para no depender de un servicio externo. Calcula el total, valida reglas mínimas y escribe una salida JSON. Es una demostración de orquestación; la carga al warehouse se desarrolla en la siguiente sección.

El contrato exige identificadores únicos, cantidades positivas y precios no negativos. Se esperan los totales 50.00, 30.00 y 15.00, con suma 95.00. Los tres registros pueden pasar por XCom por su tamaño reducido; un proceso real debería intercambiar referencias a archivos o tablas [11].

7.2. Archivo completo: s8_ventas.py

Listing 2: Pipeline autocontenido de cuatro tareas

```
from datetime import timedelta
import pendulum
from airflow.sdk import dag, task, get_current_context

@dag(
    dag_id="s8_ventas",
    schedule=None,
    start_date=pendulum.datetime(
        2026, 1, 1, tz="America/Guatemala"
    ),
    catchup=False,
    max_active_runs=1,
    default_args={
        "retries": 2,
        "retry_delay": timedelta(seconds=30),
        "execution_timeout": timedelta(minutes=5),
    },
    tags=["semana8", "sgfood"],
)
def ventas():
    @task
    def extraer():
        return [
```

```
            {"id": 1, "cantidad": 2, "precio": "25.00"},
            {"id": 2, "cantidad": 1, "precio": "30.00"},
            {"id": 3, "cantidad": 3, "precio": "5.00"},
        ]

    @task
    def transformar(filas):
        from decimal import Decimal
        return [
            {**f, "total": str(
                Decimal(f["precio"]) * f["cantidad"]
            )}
            for f in filas
        ]

    @task(retries=0)
    def validar(filas):
        from decimal import Decimal
        if not filas:
            raise ValueError("Lote vacio")
        ids = [f["id"] for f in filas]
        if len(ids) != len(set(ids)):
            raise ValueError("Identificadores duplicados")
        for f in filas:
            if f["cantidad"] <= 0 or Decimal(f["precio"]) < 0:
                raise ValueError(f"Venta invalida: {f['id']}")
        return filas

    @task
    def cargar(filas):
        import hashlib
        import json
        import logging
        from pathlib import Path
        context = get_current_context()
        clave = hashlib.sha256(
            context["run_id"].encode("utf-8")
        ).hexdigest()[:20]
```

```
62          carpeta = Path("/opt/airflow/logs/s8_resultados")
63          carpeta.mkdir(parents=True, exist_ok=True)
64          destino = carpeta / f"ventas_{clave}.json"
65          temporal = destino.with_suffix(".tmp")
66          temporal.write_text(
67              json.dumps(filas, indent=2), encoding="utf-8"
68          )
69          temporal.replace(destino)
70          logging.info("Filas=%s; salida=%s", len(filas), destino)
71          return {"filas": len(filas), "ruta": str(destino)}
72
73      cargar(validar(transformar(extraer())))
74
75  ventas()
```

7.3. Lectura del comportamiento

La cadena de llamadas establece extraer → transformar → validar → cargar. Una excepción en validar bloquea cargar. La escritura temporal seguida de reemplazo reduce la posibilidad de exponer un archivo parcialmente escrito.

La ruta usa el identificador de ejecución: repetir la tarea dentro de esa ejecución reemplaza el mismo archivo. Disparar una ejecución manual nueva crea otro archivo. Esta es idempotencia dentro de una ejecución, no deduplicación de ventas entre ejecuciones. Para un warehouse, la clave de negocio y el periodo procesado deben controlar la repetición.

El uso de la carpeta de logs como salida es una simplificación del laboratorio: ya está montada en el Compose oficial y permite inspeccionar el resultado desde el equipo anfitrión. En una solución desplegada se debe usar almacenamiento de datos dedicado y accesible a los workers.

8. Integración con dbt, SQL y el warehouse

8.1. Responsabilidades y orden de ejecución

Airflow coordina el proceso completo y dbt administra las dependencias entre sus modelos. Una propuesta ELT para SGFood es: extraer ventas, cargar raw, ejecutar dbt y verificar el mart. Conviene conservar en dbt la lógica SQL y evitar replicar manualmente cada dependencia interna en Airflow.

`dbt build` ejecuta recursos seleccionados, incluidos modelos y pruebas, siguiendo su grafo. Un fallo en una prueba puede impedir la construcción de recursos posteriores. Sin embargo, este comando no convierte todo el proyecto en una única transacción ni garantiza que consumidores no observen relaciones ya actualizadas [12].

8.2. Ejecutar un proyecto dbt preparado

Este fragmento se inserta dentro de un DAG y supone que `cargar_raw` es una tarea ya definida. El worker debe disponer del ejecutable dbt, el adaptador correspondiente, el proyecto montado y un perfil válido. Una conexión creada en Airflow no configura automáticamente `profiles.yml`.

Listing 3: Integración ilustrativa con dbt Core

```
from airflow.providers.standard.operators.bash import BashOperator

construir_modelos = BashOperator(
    task_id="construir_modelos_dbt",
    bash_command=(
        "dbt build "
        "--project-dir /opt/airflow/dbt/sgfood "
        "--profiles-dir /opt/airflow/dbt_config "
        "--target dev"
    ),
)
# cargar_raw >> construir_modelos
```

Se recomienda preparar una imagen con versiones compatibles de dbt y su adaptador, o ejecutar dbt en un entorno aislado. Para reutilizar el proyecto de la semana 7 se deben comprobar las rutas, el warehouse de destino y las credenciales disponibles dentro del contenedor. Instalar dbt únicamente en Windows no lo hace accesible al worker Linux.

8.3. Consultas y validación SQL

Los operadores comunes de SQL utilizan una conexión identificada por `conn_id`; también se requiere el provider del motor elegido. Una consulta que devuelve cero no necesariamente falla: para convertir una condición de calidad en estado de tarea se puede usar `SQLCheckOperator` [7].

```
from airflow.providers.common.sql.operators.sql import
   SQLCheckOperator

verificar_mart = SQLCheckOperator(
    task_id="verificar_mart",
    conn_id="warehouse_sgfood",
    sql="SELECT COUNT(*) > 0 FROM marts.fact_ventas",
)
construir_modelos >> verificar_mart
```

Este ejemplo presupone PostgreSQL y la tabla indicada. Comprobar que existe alguna fila es un control básico: en una carga diaria debe añadirse un filtro del lote o periodo esperado para que datos antiguos no oculten una carga vacía.

8.4. Carga idempotente al warehouse

El siguiente SQL original ilustra una carga en PostgreSQL con clave única. Debe ejecutarse mediante una conexión al warehouse, nunca contra la base de metadatos de Airflow.

Listing 4: Repetición sin duplicar la venta del ejemplo

```
CREATE SCHEMA IF NOT EXISTS raw;
CREATE TABLE IF NOT EXISTS raw.ventas (
    venta_id INTEGER PRIMARY KEY,
    fecha DATE NOT NULL,
    total NUMERIC(12,2) NOT NULL CHECK (total >= 0)
);

INSERT INTO raw.ventas (venta_id, fecha, total)
VALUES (1, DATE '2026-03-09', 50.00)
ON CONFLICT (venta_id) DO UPDATE
SET fecha = EXCLUDED.fecha, total = EXCLUDED.total;
```

Ejecutarlo dos veces debe dejar una sola venta con identificador 1. Si el origen identifica ventas por sucursal y número local, la clave deberá incluir ambos campos. Para lotes se

propone cargar a staging, validar y aplicar una transacción que actualice el destino; el diseño exacto depende del motor y del volumen.

9. Variables, conexiones y comunicación entre tareas

9.1. Configuración con Variables

Una Variable conserva configuración global consultable durante la ejecución. En el caso del curso puede representar un umbral operativo, mientras que las reglas estables se mantienen en código versionado [10].

```
from airflow.sdk import task, Variable

@task
def leer_umbral():
    return int(Variable.get("s8_min_filas", default="1"))
```

La consulta está dentro de la tarea. No se utiliza una Variable global para comunicar cuántas filas produjo una ejecución, porque varias ejecuciones podrían sobrescribir ese valor.

9.2. Conexiones y credenciales

Una conexión reúne los datos de acceso a un sistema externo. El identificador `warehouse_sgfood` permite que la tarea utilice una configuración administrada fuera de su código. Para el laboratorio, se debe registrar tipo de motor, host visible desde Docker, puerto, base y usuario con acceso al esquema requerido.

`localhost` dentro de un contenedor se refiere a ese contenedor. Si PostgreSQL está en otro servicio de Compose, se utiliza el nombre del servicio. Los permisos para escribir raw y para consultar marts deben comprobarse con el mismo usuario que utilizará el pipeline. Las credenciales se suministran mediante conexiones o mecanismos de secretos, sin incorporarlas al DAG ni a las capturas.

9.3. XCom y almacenamiento compartido

XCom permite intercambiar resultados pequeños vinculados a tareas y ejecuciones. TaskFlow lo utiliza al entregar el resultado de una tarea a otra [5], [11]. Una ruta, el identificador de un lote y un conteo son ejemplos adecuados.

Para un archivo grande, el productor escribe en almacenamiento compartido y entrega su URI. El consumidor lee esa URI con sus propios permisos. Retornar la ruta de un archivo que solo existe en el disco de un worker no garantiza que otra tarea pueda abrirlo. El contrato debe incluir formato, ubicación, periodo y condiciones de disponibilidad.

10. Monitoreo, alertas y manejo de fallos

10.1. Interpretar los estados

Estado	Interpretación operacional
`scheduled / queued`	Trabajo programado o en cola, pendiente de ejecución.
`running`	El intento está ejecutándose.
`success`	La tarea terminó correctamente.
`up_for_retry`	Se espera otro intento después de un fallo.
`failed`	La tarea falló sin recuperación automática pendiente.
`upstream_failed`	Una dependencia previa falló e impide esta ejecución.
`skipped`	La tarea fue omitida por la lógica del flujo.

Estos estados permiten distinguir un error propio de una tarea del bloqueo provocado por una dependencia [8]. En el ejemplo, una venta negativa hace fallar validar; cargar no debe continuar como si los datos fueran correctos.

10.2. Logs y evidencia

Para investigar, se identifica DAG, ejecución, tarea e intento. Después se revisan el mensaje y la excepción original. En SGFood conviene registrar periodo procesado, filas leídas, filas rechazadas y destino de salida. El log de una tarea que informa tres filas no sustituye la verificación del archivo o tabla producida.

10.3. Alertas y callbacks

Airflow permite asociar callbacks a eventos como fallo o reintento. Su ejecución depende de cambios de estado originados por la ejecución de tareas; cambiar manualmente el estado desde la interfaz no equivale a ejecutar el callback [3].

Una alerta útil debe indicar qué falló, qué ejecución está afectada, dónde están los logs y quién atiende el caso. Para enviar correo o mensajes se necesita configurar un canal y sus credenciales; declarar una función que escribe un log no envía una notificación. En clase basta con diseñar el contenido y documentar dónde se conectaría el mecanismo de aviso.

10.4. Procedimiento de recuperación

1. Localizar el primer fallo y comprobar si proviene de datos, configuración o infraestructura.

2. Corregir la causa; conservar evidencia del error y de la corrección.
3. Revisar si existe una salida parcial o una carga ya confirmada.
4. Limpiar el estado de la tarea afectada y, cuando corresponda, sus dependientes para permitir la repetición.
5. Verificar tanto el estado final como los conteos y resultados de negocio.

Marcar una tarea como exitosa no ejecuta su trabajo. Tampoco conviene agregar una tarea final con `all_done` y asumir que su éxito garantiza el del pipeline: la evaluación de las tareas hoja puede ocultar fallos anteriores si el diseño no los preserva [4].

11. Despliegue local con Docker

11.1. Preparación del entorno

Se requiere Docker con contenedores Linux y Docker Compose v2. La guía oficial indica al menos 4 GB de memoria para Docker y recomienda disponer de más recursos, idealmente 8 GB. El Compose de referencia es para aprendizaje y exploración [6].

Los comandos siguientes se ejecutan en **PowerShell**, dentro de una carpeta nueva para el laboratorio. El archivo Compose se descarga de la misma versión utilizada por los ejemplos.

Listing 5: Preparación en PowerShell

```
New-Item -ItemType Directory -Force airflow_s8
Set-Location airflow_s8
Invoke-WebRequest -Uri "https://airflow.apache.org/docs/apache-airflow
    /3.1.0/docker-compose.yaml" -OutFile docker-compose.yaml
New-Item -ItemType Directory -Force dags,logs,plugins,config
Set-Content -Path .env -Value "AIRFLOW_UID=50000" -Encoding ascii
docker compose version
docker compose config --quiet
docker compose up airflow-init
docker compose up -d
docker compose ps
```

Esperar a que la inicialización termine con código cero antes de iniciar los servicios. Copiar el código completo del ejemplo en `dags/s8_ventas.py`. La interfaz queda disponible en http://localhost:8080; el Compose oficial utiliza inicialmente usuario y contraseña `airflow`. Estas credenciales corresponden exclusivamente al entorno de aprendizaje.

11.2. Comprobación y primera ejecución

```
docker compose exec airflow-scheduler airflow version
docker compose exec airflow-scheduler airflow dags list-import-errors
docker compose logs --tail 100 airflow-dag-processor
```

La lista de errores de importación debe quedar vacía para el archivo creado. En la interfaz, localizar `s8_ventas`, revisar su grafo, habilitarlo si está pausado y disparar una ejecución manual. Abrir los logs de cada tarea y verificar cuatro estados exitosos.

La salida se encuentra en `logs/s8_resultados` del equipo anfitrión, por el montaje del Compose. Inspeccionar el JSON: debe contener tres ventas y sumar 95.00. La ruta exacta también aparece en el log de cargar.

11.3. Prueba de fallo y recuperación

Cambiar la cantidad de la primera venta a -2, guardar el archivo y esperar a que se actualice el DAG. Disparar una ejecución nueva: transformar calcula el valor, validar lo rechaza y cargar queda bloqueada. Confirmar que no se produjo el archivo para esa ejecución.

Restaurar la cantidad a 2. Para la primera práctica se recomienda disparar otra ejecución y comprobar el éxito. Si se recupera la ejecución fallida mediante limpieza de estados, hay que volver a ejecutar también extraer y transformar: sus resultados previos contienen el dato incorrecto. Después se limpian validar y cargar. La repetición debe utilizar la versión corregida del código.

11.4. Diagnóstico y cierre

Si el DAG no aparece, revisar errores de importación y el montaje de dags. Si los servicios reinician, revisar memoria y logs. Si dbt no se encuentra, comprobar la imagen del worker; si falla SQL, comprobar la conexión y el provider del motor. Un DAG importado correctamente todavía puede fallar durante su ejecución.

```
docker compose logs --tail 100 airflow-scheduler
docker compose logs --tail 100 airflow-worker
docker compose down
```

El último comando detiene y retira los contenedores y la red del proyecto; no solicita eliminar los volúmenes de datos.

12. Buenas prácticas operacionales

12.1. Diseñar para repetición y trazabilidad

Las tareas deben tolerar reejecución, utilizar periodos identificables y evitar efectos secundarios durante el procesamiento del archivo DAG. Los datos grandes se conservan fuera de XCom; los secretos se administran fuera del código [2].

Para SGFood se propone conservar una bitácora con identificador de lote, periodo, estado, conteo y versión del proyecto. Un resultado se considera reproducible cuando se conocen sus entradas, su configuración y el código que lo generó. Tener un DAG en Git es necesario para rastrear cambios, pero no recupera por sí solo un archivo de origen que fue sobrescrito.

12.2. Control de recursos y publicación

Si varias tareas consultan el mismo sistema operacional, debe limitarse su concurrencia para no saturarlo. Se puede organizar el acceso mediante pools y límites de ejecución. En el ejercicio, `max_active_runs=1` simplifica las pruebas; no elimina la necesidad de claves únicas ni transacciones.

La publicación debe depender de controles relevantes para el negocio. Como propuesta, un mart diario de SGFood podría exigir presencia del periodo esperado, ausencia de identificadores duplicados, totales no negativos y conciliación con raw. Un valor fuera de lo normal debe investigarse antes de decidir si es un error o una variación válida de ventas.

12.3. Lista de revisión del pipeline

1. Cada tarea tiene una responsabilidad, un nombre comprensible y una salida definida.
2. La frecuencia y el periodo procesado están documentados.
3. Las dependencias impiden publicar tras una validación fallida.
4. Los reintentos no producen ventas duplicadas ni archivos incompletos.
5. Los workers pueden acceder a entradas, configuración y destinos.
6. Las versiones de Airflow, providers y dbt están registradas.
7. Los logs permiten localizar el error y existe un procedimiento de recuperación.

13. Conclusiones

Orquestar un pipeline exige definir tanto su recorrido normal como su respuesta ante fallos. Los DAGs expresan dependencias; la programación determina cuándo corresponde trabajar; y las evidencias de ejecución permiten comprobar qué ocurrió. La utilidad del flujo depende de que los datos producidos cumplan el objetivo de negocio.

La integración con dbt conecta esta sesión con el modelado analítico: Airflow coordina las etapas y dbt organiza la transformación de modelos. Las conexiones, los controles SQL y una carga idempotente permiten pasar de una demostración local a un proceso de datos verificable.

En el caso SGFood, el aprendizaje se demuestra cuando el estudiante puede explicar por qué una venta inválida impide publicar, corregir la causa y repetir el trabajo sin duplicar información. Así, planificación, responsabilidad operacional, disciplina técnica y pensamiento crítico se traducen en decisiones observables.

Referencias

[1] Apache Software Foundation. «Architecture Overview,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/core-concepts/overview.html

[2] Apache Software Foundation. «Best Practices,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/best-practices.html

[3] Apache Software Foundation. «Callbacks,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/administration-and-deployment/logging-monitoring/callbacks.html

[4] Apache Software Foundation. «Dag Runs,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.2/core-concepts/dag-run.html

[5] Apache Software Foundation. «Pythonic Dags with the TaskFlow API,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/tutorial/taskflow.html

[6] Apache Software Foundation. «Running Airflow in Docker,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/howto/docker-compose/index.html

[7] Apache Software Foundation. «SQL Operators,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow-providers-common-sql/stable/operators.html

[8] Apache Software Foundation. «Tasks,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.0.3/core-concepts/tasks.html

[9] Apache Software Foundation. «Timetables,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.4/authoring-and-scheduling/timetable.html

[10] Apache Software Foundation. «Variables,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/core-concepts/variables.html

[11] Apache Software Foundation. «XComs,» visitado 9 de sep. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.1.0/core-concepts/xcoms.html

[12] dbt Labs. «About dbt build command,» visitado 9 de sep. de 2026. dirección: https://docs.getdbt.com/reference/commands/build